

```

clear all;
clc;

%% Constants

g = 9.81;
rho_lp = 784;
h0 = 11.2; % cm
A_vt = 0.0507;
A_vi = 0.164;
A_ci = 0.0678;
Pamb = 103250; % Pa

% Initial room density
P_start = 103250; % Pa
R = 287;
T_start = 23.4 + 273.15; % K
rho_start = P_start/(R*T_start);

% Final room density
P_end = 103346; % Pa
T_end = 25 + 273.15; % K
rho_end = P_end/(R*T_end);

%% Data

RPM1800 = [16.8 14 13.8 12.5 12 10.8 10.2 10 17 % open
           14.8 9.8 8 3.8 2 -1.5 -3.2 -3.2 0 % surge
           14.2 9.8 8 4.4 2.8 -0.5 -2.2 -2.4 1.8 % 1
           14.6 10 8.8 5.2 3.8 0.8 -1 -1 3.2 % 2
           15 11 10 7 5.6 3 1.5 1.4 6.5]; %3

RPM2100 = [18.8 15.2 14.8 13 12.4 10.8 10 9.5 19.4 % open
           16 9.2 6.8 1 -1.2 -6 -8.5 -8.4 -3.8 % surge
           15.6 9.2 6.8 2 -0.5 -5 -7.4 -7.2 -2 % 1
           16 10 8 3.5 1.2 -2.8 -5.2 -5.2 1 % 2
           16.8 11 9.2 5.2 3.4 -0.2 -2.2 -2.4 4.5]; % 3

RPM2400 = [21 17 16.4 18.8 12.8 10.8 9.8 9 22 % open
           17.5 8.9 5.2 -2 -5 -11.3 -14.4 -14.6 -8.4 % surge
           17 8.8 5.4 -1 -4 -10 -13.2 -13.4 -6.2 % 1
           17.4 9.4 6.8 0.5 -2.4 -8 -10.3 -11.4 -3.4 % 2
           18 8.8 8.2 2.8 0 -5 -8 -8.2 0.6]; % 3

RPM2700 = [24 18.5 17.8 14.8 13.4 9.6 9.2 8.2 25 % open
           18.8 8 3.5 -5.4 -9.2 -17 -21 -21 -12.8 % surge
           18.4 8.2 4 -4 -8 -15.2 -19.5 -19.6 -7.6 % 1
           19 9 5.6 -2 -5.8 -12.9 -17 -17.2 -7.2 % 2
           20 10.6 10.5 7.5 -2.8 -9.5 -13 -13.5 -2.2]; % 3

```

```

RPM3000 = [27 20.8 19.8 16 14 9.8 9 7.5 28.6    % open
           20 6.8 1.2 -10 -15 -24.2 -29.8 -30 -19.6 % surge
           20.4 7.8 2.4 -8 -13 -22 -27.6 -27.5 -16.2    % 1
           21.8 9 5 -4.8 -9.2 -18.2 -23 -23.8 -11    % 2
           20.4 11.8 7.8 -1 -4.8 -12.5 -17 -17.5 -3]; % 3

```

```
%% 1800 RPM open
```

```
% Converting from fluid level (cm) to pressure (Pa)
```

```

P1_open1800 = (h0 - RPM1800(1,1))*g*rho_lp*cosd(20)*(1/100);
P2_open1800 = (h0 - RPM1800(1,2))*g*rho_lp*cosd(20)*(1/100);
P3_open1800 = (h0 - RPM1800(1,3))*g*rho_lp*cosd(20)*(1/100);
P4_open1800 = (h0 - RPM1800(1,4))*g*rho_lp*cosd(20)*(1/100);
P5_open1800 = (h0 - RPM1800(1,5))*g*rho_lp*cosd(20)*(1/100);
P6_open1800 = (h0 - RPM1800(1,6))*g*rho_lp*cosd(20)*(1/100);
P7_open1800 = (h0 - RPM1800(1,7))*g*rho_lp*cosd(20)*(1/100);
P8_open1800 = (h0 - RPM1800(1,8))*g*rho_lp*cosd(20)*(1/100);
P9_open1800 = (h0 - RPM1800(1,9))*g*rho_lp*cosd(20)*(1/100);

```

```
% Mass flow rate calculation
```

```

Vth_open1800 = sqrt((2*(P8_open1800 - P9_open1800))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_open1800 = Vth_open1800*rho_end*A_vt;

```

```
% Pressure ratio calculation
```

```

Vx_open1800 = mdot_open1800/(rho_start*A_ci);
Pdyn_open1800 = 0.5*(Vx_open1800)^2*rho_start;
PT1_open1800 = P1_open1800 + Pamb + Pdyn_open1800;
PT7_open1800 = P7_open1800 + Pamb + Pdyn_open1800;
PR_open1800 = PT7_open1800/PT1_open1800;

```

```
%% 1800 RPM surge
```

```
% Converting from fluid level (cm) to pressure (Pa)
```

```

P1_surge1800 = (h0 - RPM1800(2,1))*g*rho_lp*cosd(20)*(1/100);
P2_surge1800 = (h0 - RPM1800(2,2))*g*rho_lp*cosd(20)*(1/100);
P3_surge1800 = (h0 - RPM1800(2,3))*g*rho_lp*cosd(20)*(1/100);
P4_surge1800 = (h0 - RPM1800(2,4))*g*rho_lp*cosd(20)*(1/100);
P5_surge1800 = (h0 - RPM1800(2,5))*g*rho_lp*cosd(20)*(1/100);
P6_surge1800 = (h0 - RPM1800(2,6))*g*rho_lp*cosd(20)*(1/100);
P7_surge1800 = (h0 - RPM1800(2,7))*g*rho_lp*cosd(20)*(1/100);
P8_surge1800 = (h0 - RPM1800(2,8))*g*rho_lp*cosd(20)*(1/100);
P9_surge1800 = (h0 - RPM1800(2,9))*g*rho_lp*cosd(20)*(1/100);

```

```
% Mass flow rate calculation
```

```

Vth_surge1800 = sqrt((2*(P8_surge1800 - P9_surge1800))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_surge1800 = Vth_surge1800*rho_end*A_vt;

```

```
% Pressure ratio calculation
```

```

Vx_surge1800 = mdot_surge1800/(rho_start*A_ci);
Pdyn_surge1800 = 0.5*(Vx_surge1800)^2*rho_start;

```

```

PT1_surge1800 = P1_surge1800 + Pamb + Pdyn_surge1800;
PT7_surge1800 = P7_surge1800 + Pamb + Pdyn_surge1800;
PR_surge1800 = PT7_surge1800/PT1_surge1800;

%% 1800 RPM one

% Converting from fluid level (cm) to pressure (Pa)
P1_one1800 = (h0 - RPM1800(3,1))*g*rho_lp*cosd(20)*(1/100);
P2_one1800 = (h0 - RPM1800(3,2))*g*rho_lp*cosd(20)*(1/100);
P3_one1800 = (h0 - RPM1800(3,3))*g*rho_lp*cosd(20)*(1/100);
P4_one1800 = (h0 - RPM1800(3,4))*g*rho_lp*cosd(20)*(1/100);
P5_one1800 = (h0 - RPM1800(3,5))*g*rho_lp*cosd(20)*(1/100);
P6_one1800 = (h0 - RPM1800(3,6))*g*rho_lp*cosd(20)*(1/100);
P7_one1800 = (h0 - RPM1800(3,7))*g*rho_lp*cosd(20)*(1/100);
P8_one1800 = (h0 - RPM1800(3,8))*g*rho_lp*cosd(20)*(1/100);
P9_one1800 = (h0 - RPM1800(3,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_one1800 = sqrt((2*(P8_one1800 - P9_one1800))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_one1800 = Vth_one1800*rho_end*A_vt;

% Pressure ratio calculation
Vx_one1800 = mdot_one1800/(rho_start*A_ci);
Pdyn_one1800 = 0.5*(Vx_one1800)^2*rho_start;
PT1_one1800 = P1_one1800 + Pamb + Pdyn_one1800;
PT7_one1800 = P7_one1800 + Pamb + Pdyn_one1800;
PR_one1800 = PT7_one1800/PT1_one1800;

%% 1800 RPM two

% Converting from fluid level (cm) to pressure (Pa)
P1_two1800 = (h0 - RPM1800(4,1))*g*rho_lp*cosd(20)*(1/100);
P2_two1800 = (h0 - RPM1800(4,2))*g*rho_lp*cosd(20)*(1/100);
P3_two1800 = (h0 - RPM1800(4,3))*g*rho_lp*cosd(20)*(1/100);
P4_two1800 = (h0 - RPM1800(4,4))*g*rho_lp*cosd(20)*(1/100);
P5_two1800 = (h0 - RPM1800(4,5))*g*rho_lp*cosd(20)*(1/100);
P6_two1800 = (h0 - RPM1800(4,6))*g*rho_lp*cosd(20)*(1/100);
P7_two1800 = (h0 - RPM1800(4,7))*g*rho_lp*cosd(20)*(1/100);
P8_two1800 = (h0 - RPM1800(4,8))*g*rho_lp*cosd(20)*(1/100);
P9_two1800 = (h0 - RPM1800(4,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_two1800 = sqrt((2*(P8_two1800 - P9_two1800))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_two1800 = Vth_two1800*rho_end*A_vt;

% Pressure ratio calculation
Vx_two1800 = mdot_two1800/(rho_start*A_ci);
Pdyn_two1800 = 0.5*(Vx_two1800)^2*rho_start;
PT1_two1800 = P1_two1800 + Pamb + Pdyn_two1800;
PT7_two1800 = P7_two1800 + Pamb + Pdyn_two1800;

```

```

PR_two1800 = PT7_two1800/PT1_two1800;

%% 1800 RPM three

% Converting from fluid level (cm) to pressure (Pa)
P1_three1800 = (h0 - RPM1800(5,1))*g*rho_lp*cosd(20)*(1/100);
P2_three1800 = (h0 - RPM1800(5,2))*g*rho_lp*cosd(20)*(1/100);
P3_three1800 = (h0 - RPM1800(5,3))*g*rho_lp*cosd(20)*(1/100);
P4_three1800 = (h0 - RPM1800(5,4))*g*rho_lp*cosd(20)*(1/100);
P5_three1800 = (h0 - RPM1800(5,5))*g*rho_lp*cosd(20)*(1/100);
P6_three1800 = (h0 - RPM1800(5,6))*g*rho_lp*cosd(20)*(1/100);
P7_three1800 = (h0 - RPM1800(5,7))*g*rho_lp*cosd(20)*(1/100);
P8_three1800 = (h0 - RPM1800(5,8))*g*rho_lp*cosd(20)*(1/100);
P9_three1800 = (h0 - RPM1800(5,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_three1800 = sqrt((2*(P8_three1800 - P9_three1800))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_three1800 = Vth_three1800*rho_end*A_vt;

% Pressure ratio calculation
Vx_three1800 = mdot_three1800/(rho_start*A_ci);
Pdyn_three1800 = 0.5*(Vx_three1800)^2*rho_start;
PT1_three1800 = P1_three1800 + Pamb + Pdyn_three1800;
PT7_three1800 = P7_three1800 + Pamb + Pdyn_three1800;
PR_three1800 = PT7_three1800/PT1_three1800;

%% 2100 RPM open

% Converting from fluid level (cm) to pressure (Pa)
P1_open2100 = (h0 - RPM2100(1,1))*g*rho_lp*cosd(20)*(1/100);
P2_open2100 = (h0 - RPM2100(1,2))*g*rho_lp*cosd(20)*(1/100);
P3_open2100 = (h0 - RPM2100(1,3))*g*rho_lp*cosd(20)*(1/100);
P4_open2100 = (h0 - RPM2100(1,4))*g*rho_lp*cosd(20)*(1/100);
P5_open2100 = (h0 - RPM2100(1,5))*g*rho_lp*cosd(20)*(1/100);
P6_open2100 = (h0 - RPM2100(1,6))*g*rho_lp*cosd(20)*(1/100);
P7_open2100 = (h0 - RPM2100(1,7))*g*rho_lp*cosd(20)*(1/100);
P8_open2100 = (h0 - RPM2100(1,8))*g*rho_lp*cosd(20)*(1/100);
P9_open2100 = (h0 - RPM2100(1,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_open2100 = sqrt((2*(P8_open2100 - P9_open2100))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_open2100 = Vth_open2100*rho_end*A_vt;

% Pressure ratio calculation
Vx_open2100 = mdot_open2100/(rho_start*A_ci);
Pdyn_open2100 = 0.5*(Vx_open2100)^2*rho_start;
PT1_open2100 = P1_open2100 + Pamb + Pdyn_open2100;
PT7_open2100 = P7_open2100 + Pamb + Pdyn_open2100;
PR_open2100 = PT7_open2100/PT1_open2100;

```

%% 2100 RPM surge

% Converting from fluid level (cm) to pressure (Pa)

```
P1_surge2100 = (h0 - RPM2100(2,1))*g*rho_lp*cosd(20)*(1/100);
P2_surge2100 = (h0 - RPM2100(2,2))*g*rho_lp*cosd(20)*(1/100);
P3_surge2100 = (h0 - RPM2100(2,3))*g*rho_lp*cosd(20)*(1/100);
P4_surge2100 = (h0 - RPM2100(2,4))*g*rho_lp*cosd(20)*(1/100);
P5_surge2100 = (h0 - RPM2100(2,5))*g*rho_lp*cosd(20)*(1/100);
P6_surge2100 = (h0 - RPM2100(2,6))*g*rho_lp*cosd(20)*(1/100);
P7_surge2100 = (h0 - RPM2100(2,7))*g*rho_lp*cosd(20)*(1/100);
P8_surge2100 = (h0 - RPM2100(2,8))*g*rho_lp*cosd(20)*(1/100);
P9_surge2100 = (h0 - RPM2100(2,9))*g*rho_lp*cosd(20)*(1/100);
```

% Mass flow rate calculation

```
Vth_surge2100 = sqrt((2*(P8_surge2100 - P9_surge2100))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_surge2100 = Vth_surge2100*rho_end*A_vt;
```

% Pressure ratio calculation

```
Vx_surge2100 = mdot_surge2100/(rho_start*A_ci);
Pdyn_surge2100 = 0.5*(Vx_surge2100)^2*rho_start;
PT1_surge2100 = P1_surge2100 + Pamb + Pdyn_surge2100;
PT7_surge2100 = P7_surge2100 + Pamb + Pdyn_surge2100;
PR_surge2100 = PT7_surge2100/PT1_surge2100;
```

%% 2100 RPM one

% Converting from fluid level (cm) to pressure (Pa)

```
P1_one2100 = (h0 - RPM2100(3,1))*g*rho_lp*cosd(20)*(1/100);
P2_one2100 = (h0 - RPM2100(3,2))*g*rho_lp*cosd(20)*(1/100);
P3_one2100 = (h0 - RPM2100(3,3))*g*rho_lp*cosd(20)*(1/100);
P4_one2100 = (h0 - RPM2100(3,4))*g*rho_lp*cosd(20)*(1/100);
P5_one2100 = (h0 - RPM2100(3,5))*g*rho_lp*cosd(20)*(1/100);
P6_one2100 = (h0 - RPM2100(3,6))*g*rho_lp*cosd(20)*(1/100);
P7_one2100 = (h0 - RPM2100(3,7))*g*rho_lp*cosd(20)*(1/100);
P8_one2100 = (h0 - RPM2100(3,8))*g*rho_lp*cosd(20)*(1/100);
P9_one2100 = (h0 - RPM2100(3,9))*g*rho_lp*cosd(20)*(1/100);
```

% Mass flow rate calculation

```
Vth_one2100 = sqrt((2*(P8_one2100 - P9_one2100))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_one2100 = Vth_one2100*rho_end*A_vt;
```

% Pressure ratio calculation

```
Vx_one2100 = mdot_one2100/(rho_start*A_ci);
Pdyn_one2100 = 0.5*(Vx_one2100)^2*rho_start;
PT1_one2100 = P1_one2100 + Pamb + Pdyn_one2100;
PT7_one2100 = P7_one2100 + Pamb + Pdyn_one2100;
PR_one2100 = PT7_one2100/PT1_one2100;
```

%% 2100 RPM two

```

% Converting from fluid level (cm) to pressure (Pa)
P1_two2100 = (h0 - RPM2100(4,1))*g*rho_lp*cosd(20)*(1/100);
P2_two2100 = (h0 - RPM2100(4,2))*g*rho_lp*cosd(20)*(1/100);
P3_two2100 = (h0 - RPM2100(4,3))*g*rho_lp*cosd(20)*(1/100);
P4_two2100 = (h0 - RPM2100(4,4))*g*rho_lp*cosd(20)*(1/100);
P5_two2100 = (h0 - RPM2100(4,5))*g*rho_lp*cosd(20)*(1/100);
P6_two2100 = (h0 - RPM2100(4,6))*g*rho_lp*cosd(20)*(1/100);
P7_two2100 = (h0 - RPM2100(4,7))*g*rho_lp*cosd(20)*(1/100);
P8_two2100 = (h0 - RPM2100(4,8))*g*rho_lp*cosd(20)*(1/100);
P9_two2100 = (h0 - RPM2100(4,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_two2100 = sqrt((2*(P8_two2100 - P9_two2100))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_two2100 = Vth_two2100*rho_end*A_vt;

% Pressure ratio calculation
Vx_two2100 = mdot_two2100/(rho_start*A_ci);
Pdyn_two2100 = 0.5*(Vx_two2100)^2*rho_start;
PT1_two2100 = P1_two2100 + Pamb + Pdyn_two2100;
PT7_two2100 = P7_two2100 + Pamb + Pdyn_two2100;
PR_two2100 = PT7_two2100/PT1_two2100;

%% 2100 RPM three

% Converting from fluid level (cm) to pressure (Pa)
P1_three2100 = (h0 - RPM2100(5,1))*g*rho_lp*cosd(20)*(1/100);
P2_three2100 = (h0 - RPM2100(5,2))*g*rho_lp*cosd(20)*(1/100);
P3_three2100 = (h0 - RPM2100(5,3))*g*rho_lp*cosd(20)*(1/100);
P4_three2100 = (h0 - RPM2100(5,4))*g*rho_lp*cosd(20)*(1/100);
P5_three2100 = (h0 - RPM2100(5,5))*g*rho_lp*cosd(20)*(1/100);
P6_three2100 = (h0 - RPM2100(5,6))*g*rho_lp*cosd(20)*(1/100);
P7_three2100 = (h0 - RPM2100(5,7))*g*rho_lp*cosd(20)*(1/100);
P8_three2100 = (h0 - RPM2100(5,8))*g*rho_lp*cosd(20)*(1/100);
P9_three2100 = (h0 - RPM2100(5,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_three2100 = sqrt((2*(P8_three2100 - P9_three2100))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_three2100 = Vth_three2100*rho_end*A_vt;

% Pressure ratio calculation
Vx_three2100 = mdot_three2100/(rho_start*A_ci);
Pdyn_three2100 = 0.5*(Vx_three2100)^2*rho_start;
PT1_three2100 = P1_three2100 + Pamb + Pdyn_three2100;
PT7_three2100 = P7_three2100 + Pamb + Pdyn_three2100;
PR_three2100 = PT7_three2100/PT1_three2100;

%% 2400 RPM open

% Converting from fluid level (cm) to pressure (Pa)

```

```

P1_open2400 = (h0 - RPM2400(1,1))*g*rho_lp*cosd(20)*(1/100);
P2_open2400 = (h0 - RPM2400(1,2))*g*rho_lp*cosd(20)*(1/100);
P3_open2400 = (h0 - RPM2400(1,3))*g*rho_lp*cosd(20)*(1/100);
P4_open2400 = (h0 - RPM2400(1,4))*g*rho_lp*cosd(20)*(1/100);
P5_open2400 = (h0 - RPM2400(1,5))*g*rho_lp*cosd(20)*(1/100);
P6_open2400 = (h0 - RPM2400(1,6))*g*rho_lp*cosd(20)*(1/100);
P7_open2400 = (h0 - RPM2400(1,7))*g*rho_lp*cosd(20)*(1/100);
P8_open2400 = (h0 - RPM2400(1,8))*g*rho_lp*cosd(20)*(1/100);
P9_open2400 = (h0 - RPM2400(1,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_open2400 = sqrt((2*(P8_open2400 - P9_open2400))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_open2400 = Vth_open2400*rho_end*A_vt;

% Pressure ratio calculation
Vx_open2400 = mdot_open2400/(rho_start*A_ci);
Pdyn_open2400 = 0.5*(Vx_open2400)^2*rho_start;
PT1_open2400 = P1_open2400 + Pamb + Pdyn_open2400;
PT7_open2400 = P7_open2400 + Pamb + Pdyn_open2400;
PR_open2400 = PT7_open2400/PT1_open2400;

%% 2400 RPM surge

% Converting from fluid level (cm) to pressure (Pa)
P1_surge2400 = (h0 - RPM2400(2,1))*g*rho_lp*cosd(20)*(1/100);
P2_surge2400 = (h0 - RPM2400(2,2))*g*rho_lp*cosd(20)*(1/100);
P3_surge2400 = (h0 - RPM2400(2,3))*g*rho_lp*cosd(20)*(1/100);
P4_surge2400 = (h0 - RPM2400(2,4))*g*rho_lp*cosd(20)*(1/100);
P5_surge2400 = (h0 - RPM2400(2,5))*g*rho_lp*cosd(20)*(1/100);
P6_surge2400 = (h0 - RPM2400(2,6))*g*rho_lp*cosd(20)*(1/100);
P7_surge2400 = (h0 - RPM2400(2,7))*g*rho_lp*cosd(20)*(1/100);
P8_surge2400 = (h0 - RPM2400(2,8))*g*rho_lp*cosd(20)*(1/100);
P9_surge2400 = (h0 - RPM2400(2,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_surge2400 = sqrt((2*(P8_surge2400 - P9_surge2400))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_surge2400 = Vth_surge2400*rho_end*A_vt;

% Pressure ratio calculation
Vx_surge2400 = mdot_surge2400/(rho_start*A_ci);
Pdyn_surge2400 = 0.5*(Vx_surge2400)^2*rho_start;
PT1_surge2400 = P1_surge2400 + Pamb + Pdyn_surge2400;
PT7_surge2400 = P7_surge2400 + Pamb + Pdyn_surge2400;
PR_surge2400 = PT7_surge2400/PT1_surge2400;

%% 2400 RPM one

% Converting from fluid level (cm) to pressure (Pa)
P1_one2400 = (h0 - RPM2400(3,1))*g*rho_lp*cosd(20)*(1/100);
P2_one2400 = (h0 - RPM2400(3,2))*g*rho_lp*cosd(20)*(1/100);

```



```

P3_one2400 = (h0 - RPM2400(3,3))*g*rho_lp*cosd(20)*(1/100);
P4_one2400 = (h0 - RPM2400(3,4))*g*rho_lp*cosd(20)*(1/100);
P5_one2400 = (h0 - RPM2400(3,5))*g*rho_lp*cosd(20)*(1/100);
P6_one2400 = (h0 - RPM2400(3,6))*g*rho_lp*cosd(20)*(1/100);
P7_one2400 = (h0 - RPM2400(3,7))*g*rho_lp*cosd(20)*(1/100);
P8_one2400 = (h0 - RPM2400(3,8))*g*rho_lp*cosd(20)*(1/100);
P9_one2400 = (h0 - RPM2400(3,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_one2400 = sqrt((2*(P8_one2400 - P9_one2400))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_one2400 = Vth_one2400*rho_end*A_vt;

% Pressure ratio calculation
Vx_one2400 = mdot_one2400/(rho_start*A_ci);
Pdyn_one2400 = 0.5*(Vx_one2400)^2*rho_start;
PT1_one2400 = P1_one2400 + Pamb + Pdyn_one2400;
PT7_one2400 = P7_one2400 + Pamb + Pdyn_one2400;
PR_one2400 = PT7_one2400/PT1_one2400;

%% 2400 RPM two

% Converting from fluid level (cm) to pressure (Pa)
P1_two2400 = (h0 - RPM2400(4,1))*g*rho_lp*cosd(20)*(1/100);
P2_two2400 = (h0 - RPM2400(4,2))*g*rho_lp*cosd(20)*(1/100);
P3_two2400 = (h0 - RPM2400(4,3))*g*rho_lp*cosd(20)*(1/100);
P4_two2400 = (h0 - RPM2400(4,4))*g*rho_lp*cosd(20)*(1/100);
P5_two2400 = (h0 - RPM2400(4,5))*g*rho_lp*cosd(20)*(1/100);
P6_two2400 = (h0 - RPM2400(4,6))*g*rho_lp*cosd(20)*(1/100);
P7_two2400 = (h0 - RPM2400(4,7))*g*rho_lp*cosd(20)*(1/100);
P8_two2400 = (h0 - RPM2400(4,8))*g*rho_lp*cosd(20)*(1/100);
P9_two2400 = (h0 - RPM2400(4,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_two2400 = sqrt((2*(P8_two2400 - P9_two2400))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_two2400 = Vth_two2400*rho_end*A_vt;

% Pressure ratio calculation
Vx_two2400 = mdot_two2400/(rho_start*A_ci);
Pdyn_two2400 = 0.5*(Vx_two2400)^2*rho_start;
PT1_two2400 = P1_two2400 + Pamb + Pdyn_two2400;
PT7_two2400 = P7_two2400 + Pamb + Pdyn_two2400;
PR_two2400 = PT7_two2400/PT1_two2400;

%% 2400 RPM three

% Converting from fluid level (cm) to pressure (Pa)
P1_three2400 = (h0 - RPM2400(5,1))*g*rho_lp*cosd(20)*(1/100);
P2_three2400 = (h0 - RPM2400(5,2))*g*rho_lp*cosd(20)*(1/100);
P3_three2400 = (h0 - RPM2400(5,3))*g*rho_lp*cosd(20)*(1/100);
P4_three2400 = (h0 - RPM2400(5,4))*g*rho_lp*cosd(20)*(1/100);

```



```

P5_three2400 = (h0 - RPM2400(5,5))*g*rho_lp*cosd(20)*(1/100);
P6_three2400 = (h0 - RPM2400(5,6))*g*rho_lp*cosd(20)*(1/100);
P7_three2400 = (h0 - RPM2400(5,7))*g*rho_lp*cosd(20)*(1/100);
P8_three2400 = (h0 - RPM2400(5,8))*g*rho_lp*cosd(20)*(1/100);
P9_three2400 = (h0 - RPM2400(5,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_three2400 = sqrt((2*(P8_three2400 - P9_three2400))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_three2400 = Vth_three2400*rho_end*A_vt;

% Pressure ratio calculation
Vx_three2400 = mdot_three2400/(rho_start*A_ci);
Pdyn_three2400 = 0.5*(Vx_three2400)^2*rho_start;
PT1_three2400 = P1_three2400 + Pamb + Pdyn_three2400;
PT7_three2400 = P7_three2400 + Pamb + Pdyn_three2400;
PR_three2400 = PT7_three2400/PT1_three2400;

%% 2700 RPM open

% Converting from fluid level (cm) to pressure (Pa)
P1_open2700 = (h0 - RPM2700(1,1))*g*rho_lp*cosd(20)*(1/100);
P2_open2700 = (h0 - RPM2700(1,2))*g*rho_lp*cosd(20)*(1/100);
P3_open2700 = (h0 - RPM2700(1,3))*g*rho_lp*cosd(20)*(1/100);
P4_open2700 = (h0 - RPM2700(1,4))*g*rho_lp*cosd(20)*(1/100);
P5_open2700 = (h0 - RPM2700(1,5))*g*rho_lp*cosd(20)*(1/100);
P6_open2700 = (h0 - RPM2700(1,6))*g*rho_lp*cosd(20)*(1/100);
P7_open2700 = (h0 - RPM2700(1,7))*g*rho_lp*cosd(20)*(1/100);
P8_open2700 = (h0 - RPM2700(1,8))*g*rho_lp*cosd(20)*(1/100);
P9_open2700 = (h0 - RPM2700(1,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_open2700 = sqrt((2*(P8_open2700 - P9_open2700))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_open2700 = Vth_open2700*rho_end*A_vt;

% Pressure ratio calculation
Vx_open2700 = mdot_open2700/(rho_start*A_ci);
Pdyn_open2700 = 0.5*(Vx_open2700)^2*rho_start;
PT1_open2700 = P1_open2700 + Pamb + Pdyn_open2700;
PT7_open2700 = P7_open2700 + Pamb + Pdyn_open2700;
PR_open2700 = PT7_open2700/PT1_open2700;

%% 2700 RPM surge

% Converting from fluid level (cm) to pressure (Pa)
P1_surge2700 = (h0 - RPM2700(2,1))*g*rho_lp*cosd(20)*(1/100);
P2_surge2700 = (h0 - RPM2700(2,2))*g*rho_lp*cosd(20)*(1/100);
P3_surge2700 = (h0 - RPM2700(2,3))*g*rho_lp*cosd(20)*(1/100);
P4_surge2700 = (h0 - RPM2700(2,4))*g*rho_lp*cosd(20)*(1/100);
P5_surge2700 = (h0 - RPM2700(2,5))*g*rho_lp*cosd(20)*(1/100);
P6_surge2700 = (h0 - RPM2700(2,6))*g*rho_lp*cosd(20)*(1/100);

```

```

P7_surge2700 = (h0 - RPM2700(2,7))*g*rho_lp*cosd(20)*(1/100);
P8_surge2700 = (h0 - RPM2700(2,8))*g*rho_lp*cosd(20)*(1/100);
P9_surge2700 = (h0 - RPM2700(2,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_surge2700 = sqrt((2*(P8_surge2700 - P9_surge2700))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_surge2700 = Vth_surge2700*rho_end*A_vt;

% Pressure ratio calculation
Vx_surge2700 = mdot_surge2700/(rho_start*A_ci);
Pdyn_surge2700 = 0.5*(Vx_surge2700)^2*rho_start;
PT1_surge2700 = P1_surge2700 + Pamb + Pdyn_surge2700;
PT7_surge2700 = P7_surge2700 + Pamb + Pdyn_surge2700;
PR_surge2700 = PT7_surge2700/PT1_surge2700;

%% 2700 RPM one

% Converting from fluid level (cm) to pressure (Pa)
P1_one2700 = (h0 - RPM2700(3,1))*g*rho_lp*cosd(20)*(1/100);
P2_one2700 = (h0 - RPM2700(3,2))*g*rho_lp*cosd(20)*(1/100);
P3_one2700 = (h0 - RPM2700(3,3))*g*rho_lp*cosd(20)*(1/100);
P4_one2700 = (h0 - RPM2700(3,4))*g*rho_lp*cosd(20)*(1/100);
P5_one2700 = (h0 - RPM2700(3,5))*g*rho_lp*cosd(20)*(1/100);
P6_one2700 = (h0 - RPM2700(3,6))*g*rho_lp*cosd(20)*(1/100);
P7_one2700 = (h0 - RPM2700(3,7))*g*rho_lp*cosd(20)*(1/100);
P8_one2700 = (h0 - RPM2700(3,8))*g*rho_lp*cosd(20)*(1/100);
P9_one2700 = (h0 - RPM2700(3,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_one2700 = sqrt((2*(P8_one2700 - P9_one2700))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_one2700 = Vth_one2700*rho_end*A_vt;

% Pressure ratio calculation
Vx_one2700 = mdot_one2700/(rho_start*A_ci);
Pdyn_one2700 = 0.5*(Vx_one2700)^2*rho_start;
PT1_one2700 = P1_one2700 + Pamb + Pdyn_one2700;
PT7_one2700 = P7_one2700 + Pamb + Pdyn_one2700;
PR_one2700 = PT7_one2700/PT1_one2700;

%% 2700 RPM two

% Converting from fluid level (cm) to pressure (Pa)
P1_two2700 = (h0 - RPM2700(4,1))*g*rho_lp*cosd(20)*(1/100);
P2_two2700 = (h0 - RPM2700(4,2))*g*rho_lp*cosd(20)*(1/100);
P3_two2700 = (h0 - RPM2700(4,3))*g*rho_lp*cosd(20)*(1/100);
P4_two2700 = (h0 - RPM2700(4,4))*g*rho_lp*cosd(20)*(1/100);
P5_two2700 = (h0 - RPM2700(4,5))*g*rho_lp*cosd(20)*(1/100);
P6_two2700 = (h0 - RPM2700(4,6))*g*rho_lp*cosd(20)*(1/100);
P7_two2700 = (h0 - RPM2700(4,7))*g*rho_lp*cosd(20)*(1/100);
P8_two2700 = (h0 - RPM2700(4,8))*g*rho_lp*cosd(20)*(1/100);

```

```

P9_two2700 = (h0 - RPM2700(4,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_two2700 = sqrt((2*(P8_two2700 - P9_two2700))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_two2700 = Vth_two2700*rho_end*A_vt;

% Pressure ratio calculation
Vx_two2700 = mdot_two2700/(rho_start*A_ci);
Pdyn_two2700 = 0.5*(Vx_two2700)^2*rho_start;
PT1_two2700 = P1_two2700 + Pamb + Pdyn_two2700;
PT7_two2700 = P7_two2700 + Pamb + Pdyn_two2700;
PR_two2700 = PT7_two2700/PT1_two2700;

%% 2700 RPM three

% Converting from fluid level (cm) to pressure (Pa)
P1_three2700 = (h0 - RPM2700(5,1))*g*rho_lp*cosd(20)*(1/100);
P2_three2700 = (h0 - RPM2700(5,2))*g*rho_lp*cosd(20)*(1/100);
P3_three2700 = (h0 - RPM2700(5,3))*g*rho_lp*cosd(20)*(1/100);
P4_three2700 = (h0 - RPM2700(5,4))*g*rho_lp*cosd(20)*(1/100);
P5_three2700 = (h0 - RPM2700(5,5))*g*rho_lp*cosd(20)*(1/100);
P6_three2700 = (h0 - RPM2700(5,6))*g*rho_lp*cosd(20)*(1/100);
P7_three2700 = (h0 - RPM2700(5,7))*g*rho_lp*cosd(20)*(1/100);
P8_three2700 = (h0 - RPM2700(5,8))*g*rho_lp*cosd(20)*(1/100);
P9_three2700 = (h0 - RPM2700(5,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_three2700 = sqrt((2*(P8_three2700 - P9_three2700))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_three2700 = Vth_three2700*rho_end*A_vt;

% Pressure ratio calculation
Vx_three2700 = mdot_three2700/(rho_start*A_ci);
Pdyn_three2700 = 0.5*(Vx_three2700)^2*rho_start;
PT1_three2700 = P1_three2700 + Pamb + Pdyn_three2700;
PT7_three2700 = P7_three2700 + Pamb + Pdyn_three2700;
PR_three2700 = PT7_three2700/PT1_three2700;

%% 3000 RPM open

% Converting from fluid level (cm) to pressure (Pa)
P1_open3000 = (h0 - RPM3000(1,1))*g*rho_lp*cosd(20)*(1/100);
P2_open3000 = (h0 - RPM3000(1,2))*g*rho_lp*cosd(20)*(1/100);
P3_open3000 = (h0 - RPM3000(1,3))*g*rho_lp*cosd(20)*(1/100);
P4_open3000 = (h0 - RPM3000(1,4))*g*rho_lp*cosd(20)*(1/100);
P5_open3000 = (h0 - RPM3000(1,5))*g*rho_lp*cosd(20)*(1/100);
P6_open3000 = (h0 - RPM3000(1,6))*g*rho_lp*cosd(20)*(1/100);
P7_open3000 = (h0 - RPM3000(1,7))*g*rho_lp*cosd(20)*(1/100);
P8_open3000 = (h0 - RPM3000(1,8))*g*rho_lp*cosd(20)*(1/100);
P9_open3000 = (h0 - RPM3000(1,9))*g*rho_lp*cosd(20)*(1/100);

```

% Mass flow rate calculation

```
Vth_open3000 = sqrt((2*(P8_open3000 - P9_open3000))/(rho_end*(1 - (A_vt/A_vi)^2)));  
mdot_open3000 = Vth_open3000*rho_end*A_vt;
```

% Pressure ratio calculation

```
Vx_open3000 = mdot_open3000/(rho_start*A_ci);  
Pdyn_open3000 = 0.5*(Vx_open3000)^2*rho_start;  
PT1_open3000 = P1_open3000 + Pamb + Pdyn_open3000;  
PT7_open3000 = P7_open3000 + Pamb + Pdyn_open3000;  
PR_open3000 = PT7_open3000/PT1_open3000;
```

%% 3000 RPM surge

% Converting from fluid level (cm) to pressure (Pa)

```
P1_surge3000 = (h0 - RPM3000(2,1))*g*rho_lp*cosd(20)*(1/100);  
P2_surge3000 = (h0 - RPM3000(2,2))*g*rho_lp*cosd(20)*(1/100);  
P3_surge3000 = (h0 - RPM3000(2,3))*g*rho_lp*cosd(20)*(1/100);  
P4_surge3000 = (h0 - RPM3000(2,4))*g*rho_lp*cosd(20)*(1/100);  
P5_surge3000 = (h0 - RPM3000(2,5))*g*rho_lp*cosd(20)*(1/100);  
P6_surge3000 = (h0 - RPM3000(2,6))*g*rho_lp*cosd(20)*(1/100);  
P7_surge3000 = (h0 - RPM3000(2,7))*g*rho_lp*cosd(20)*(1/100);  
P8_surge3000 = (h0 - RPM3000(2,8))*g*rho_lp*cosd(20)*(1/100);  
P9_surge3000 = (h0 - RPM3000(2,9))*g*rho_lp*cosd(20)*(1/100);
```

% Mass flow rate calculation

```
Vth_surge3000 = sqrt((2*(P8_surge3000 - P9_surge3000))/(rho_end*(1 - (A_vt/A_vi)^2)));  
mdot_surge3000 = Vth_surge3000*rho_end*A_vt;
```

% Pressure ratio calculation

```
Vx_surge3000 = mdot_surge3000/(rho_start*A_ci);  
Pdyn_surge3000 = 0.5*(Vx_surge3000)^2*rho_start;  
PT1_surge3000 = P1_surge3000 + Pamb + Pdyn_surge3000;  
PT7_surge3000 = P7_surge3000 + Pamb + Pdyn_surge3000;  
PR_surge3000 = PT7_surge3000/PT1_surge3000;
```

%% 3000 RPM one

% Converting from fluid level (cm) to pressure (Pa)

```
P1_one3000 = (h0 - RPM3000(3,1))*g*rho_lp*cosd(20)*(1/100);  
P2_one3000 = (h0 - RPM3000(3,2))*g*rho_lp*cosd(20)*(1/100);  
P3_one3000 = (h0 - RPM3000(3,3))*g*rho_lp*cosd(20)*(1/100);  
P4_one3000 = (h0 - RPM3000(3,4))*g*rho_lp*cosd(20)*(1/100);  
P5_one3000 = (h0 - RPM3000(3,5))*g*rho_lp*cosd(20)*(1/100);  
P6_one3000 = (h0 - RPM3000(3,6))*g*rho_lp*cosd(20)*(1/100);  
P7_one3000 = (h0 - RPM3000(3,7))*g*rho_lp*cosd(20)*(1/100);  
P8_one3000 = (h0 - RPM3000(3,8))*g*rho_lp*cosd(20)*(1/100);  
P9_one3000 = (h0 - RPM3000(3,9))*g*rho_lp*cosd(20)*(1/100);
```

% Mass flow rate calculation

```
Vth_one3000 = sqrt((2*(P8_one3000 - P9_one3000))/(rho_end*(1 - (A_vt/A_vi)^2)));
```

```

mdot_one3000 = Vth_one3000*rho_end*A_vt;

% Pressure ratio calculation
Vx_one3000 = mdot_one3000/(rho_start*A_ci);
Pdyn_one3000 = 0.5*(Vx_one3000)^2*rho_start;
PT1_one3000 = P1_one3000 + Pamb + Pdyn_one3000;
PT7_one3000 = P7_one3000 + Pamb + Pdyn_one3000;
PR_one3000 = PT7_one3000/PT1_one3000;

%% 3000 RPM two

% Converting from fluid level (cm) to pressure (Pa)
P1_two3000 = (h0 - RPM3000(4,1))*g*rho_lp*cosd(20)*(1/100);
P2_two3000 = (h0 - RPM3000(4,2))*g*rho_lp*cosd(20)*(1/100);
P3_two3000 = (h0 - RPM3000(4,3))*g*rho_lp*cosd(20)*(1/100);
P4_two3000 = (h0 - RPM3000(4,4))*g*rho_lp*cosd(20)*(1/100);
P5_two3000 = (h0 - RPM3000(4,5))*g*rho_lp*cosd(20)*(1/100);
P6_two3000 = (h0 - RPM3000(4,6))*g*rho_lp*cosd(20)*(1/100);
P7_two3000 = (h0 - RPM3000(4,7))*g*rho_lp*cosd(20)*(1/100);
P8_two3000 = (h0 - RPM3000(4,8))*g*rho_lp*cosd(20)*(1/100);
P9_two3000 = (h0 - RPM3000(4,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_two3000 = sqrt((2*(P8_two3000 - P9_two3000))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_two3000 = Vth_two3000*rho_end*A_vt;

% Pressure ratio calculation
Vx_two3000 = mdot_two3000/(rho_start*A_ci);
Pdyn_two3000 = 0.5*(Vx_two3000)^2*rho_start;
PT1_two3000 = P1_two3000 + Pamb + Pdyn_two3000;
PT7_two3000 = P7_two3000 + Pamb + Pdyn_two3000;
PR_two3000 = PT7_two3000/PT1_two3000;

%% 3000 RPM three

% Converting from fluid level (cm) to pressure (Pa)
P1_three3000 = (h0 - RPM3000(5,1))*g*rho_lp*cosd(20)*(1/100);
P2_three3000 = (h0 - RPM3000(5,2))*g*rho_lp*cosd(20)*(1/100);
P3_three3000 = (h0 - RPM3000(5,3))*g*rho_lp*cosd(20)*(1/100);
P4_three3000 = (h0 - RPM3000(5,4))*g*rho_lp*cosd(20)*(1/100);
P5_three3000 = (h0 - RPM3000(5,5))*g*rho_lp*cosd(20)*(1/100);
P6_three3000 = (h0 - RPM3000(5,6))*g*rho_lp*cosd(20)*(1/100);
P7_three3000 = (h0 - RPM3000(5,7))*g*rho_lp*cosd(20)*(1/100);
P8_three3000 = (h0 - RPM3000(5,8))*g*rho_lp*cosd(20)*(1/100);
P9_three3000 = (h0 - RPM3000(5,9))*g*rho_lp*cosd(20)*(1/100);

% Mass flow rate calculation
Vth_three3000 = sqrt((2*(P8_three3000 - P9_three3000))/(rho_end*(1 - (A_vt/A_vi)^2)));
mdot_three3000 = Vth_three3000*rho_end*A_vt;

```

% Pressure ratio calculation

```
Vx_three3000 = mdot_three3000/(rho_start*A_ci);  
Pdyn_three3000 = 0.5*(Vx_three3000)^2*rho_start;  
PT1_three3000 = P1_three3000 + Pamb + Pdyn_three3000;  
PT7_three3000 = P7_three3000 + Pamb + Pdyn_three3000;  
PR_three3000 = PT7_three3000/PT1_three3000;
```

%% Data for Excel

```
% PT7_3000 = [PT7_open3000 PT7_surge3000 PT7_one3000 PT7_two3000 PT7_three3000];  
% PT1_3000 = [PT1_open3000 PT1_surge3000 PT1_one3000 PT1_two3000 PT1_three3000];  
%  
% PT7_2700 = [PT7_open2700 PT7_surge2700 PT7_one2700 PT7_two2700 PT7_three2700];  
% PT1_2700 = [PT1_open2700 PT1_surge2700 PT1_one2700 PT1_two2700 PT1_three2700];  
%  
% PT7_2400 = [PT7_open2400 PT7_surge2400 PT7_one2400 PT7_two2400 PT7_three2400];  
% PT1_2400 = [PT1_open2400 PT1_surge2400 PT1_one2400 PT1_two2400 PT1_three2400];  
%  
% PT7_2100 = [PT7_open2100 PT7_surge2100 PT7_one2100 PT7_two2100 PT7_three2100];  
% PT1_2100 = [PT1_open2100 PT1_surge2100 PT1_one2100 PT1_two2100 PT1_three2100];  
%  
% PT7_1800 = [PT7_open1800 PT7_surge1800 PT7_one1800 PT7_two1800 PT7_three1800];  
% PT1_1800 = [PT1_open1800 PT1_surge1800 PT1_one1800 PT1_two1800 PT1_three1800];
```

```
% eta_3000 = [0.753950869 0.837801845 0.835158794 0.85299463 0.811775005];  
% eta_2700 = [0.836392883 0.857478927 1.006173817 0.880775319 0.894451174];  
% eta_2400 = [0.906834787 0.953418798 0.97295546 0.956342574 0.983475665];  
% eta_2100 = [1.049370309 0.968765177 0.969454976 1.007303523 1.036113428];  
% eta_1800 = [1.250726873 1.056330008 1.149852797 1.114351963 1.179321551];
```

%% Graph

```
% mdot_1800 = [mdot_open1800 mdot_one1800 mdot_two1800 mdot_three1800 mdot_surge1800];  
mdot_1800 = [1.863819102,1.590888808,1.443708069,1.443708069,1.260171715];  
PR_1800 = [1.004623993,1.009453993,1.010925794,1.011482874,1.012613891];  
  
mdot_2100 = [2.216523478,1.850458219,1.754084792,1.606410042,1.510892809];  
PR_2100 = [1.006166251,1.013312069,1.014849627,1.016112879,1.017172975];  
  
mdot_2400 = [2.539957294,2.089758376,1.992506432,1.890257573,1.754084792];  
PR_2400 = [1.007849558,1.018216902,1.019406546,1.021159411,1.022367988];  
  
mdot_2700 = [2.887416137,2.368066861,2.227689913,2.440312033,2.017259013];  
PR_2700 = [1.010377434,1.02312892,1.025228019,1.026525456,1.027908739];  
  
mdot_3000 = [3.235906513,2.682493879,2.520343431,2.368066861,2.271806868];  
PR_3000 = [1.012624325,1.02618396,1.031418561,1.033651499,1.034917203];
```

% Efficiency datasets

```
eta_3000 = [0.561834252,0.660960546,0.735023244,0.754381559,0.786631332];
```

```

eta_2700 = [0.590228165 0.724058053 0.750172902 0.867735251 0.775658322];
eta_2400 = [0.584528938 0.774561197 0.781952517 0.809690184 0.818356162];
eta_2100 = [0.61466189 0.735349673 0.766637775 0.776150062 0.820462602];
eta_1800 = [0.651448819 0.745339588 0.792525945 0.838187098 0.852369702];

% Store datasets in cell arrays
X = {mdot_1800, mdot_2100, mdot_2400, mdot_2700, mdot_3000};
Y = {PR_1800, PR_2100, PR_2400, PR_2700, PR_3000};
Z = {eta_1800, eta_2100, eta_2400, eta_2700, eta_3000};

% Define anomaly points manually
anomalies = {
    [], % No anomalies for dataset 1
    [], % No anomalies for dataset 2
    [], % No anomalies for dataset 3
    [2.440312033, 1.026525456], % Anomaly in dataset 4
    [2.682493879, 1.02618396] % Anomaly in dataset 5
};

% Define polynomial order for best fit (adjustable)
order = 3;

% Colors for each dataset
colors = lines(5);

figure;
hold on;
grid on;
grid minor;

scatter_handles = gobjects(1, 5); % Store scatter plot handles for legend

% Store surge points
mdot_surge = zeros(1, 5);
PR_surge = zeros(1, 5);

% Loop through each dataset
for i = 1:5
    % Extract original dataset
    x_orig = X{i};
    y_orig = Y{i};

    % Identify and remove anomalies for the best-fit curve
    if ~isempty(anomalies{i})
        anomaly_mask = false(size(x_orig)); % Initialize mask

        % Loop through anomalies and mark them for exclusion
        for j = 1:size(anomalies{i}, 1)
            anomaly_mask = anomaly_mask | (x_orig == ...
                anomalies{i}(j,1) & y_orig == anomalies{i}(j,2));
        end
    end
end

```



```

end

% Filter out anomalies for best-fit curve
x_filtered = x_orig(~anomaly_mask);
y_filtered = y_orig(~anomaly_mask);
else
    x_filtered = x_orig;
    y_filtered = y_orig;
end

% Compute polynomial fit (only for non-anomalous points)
coeffs = polyfit(x_filtered, y_filtered, order);

% Generate smooth x values for the curve
x_smooth = linspace(min(x_filtered), max(x_filtered), 100);
y_smooth = polyval(coeffs, x_smooth);

% Plot all original data points (including anomalies)
scatter_handles(i) = scatter(x_orig, y_orig, 50, colors(i, :), 'filled');

% Plot best-fit curve (excluding anomalies)
plot(x_smooth, y_smooth, 'Color', colors(i, :), 'LineWidth', 2);

% Store surge points (last point in each dataset)
mdot_surge(i) = x_orig(end);
PR_surge(i) = y_orig(end);
end

% Plot surge line (connecting last points of each dataset)
plot(mdot_surge, PR_surge, 'k--', 'LineWidth', 2, 'DisplayName', 'Surge Line');
scatter(mdot_surge, PR_surge, 80, 'k', 'filled', 'DisplayName', 'Surge Points');

% Create efficiency contour plot with thin, dashed blue lines
[Md, Pr] = meshgrid(linspace(min(cellfun(@min, X)), max(cellfun(@max, X)), 50), ...
    linspace(min(cellfun(@min, Y)), max(cellfun(@max, Y)), 50));
Eta = griddata(cell2mat(X'), cell2mat(Y'), cell2mat(Z'), Md, Pr, 'cubic');

contour_levels = linspace(min([eta_1800, eta_2100, eta_2400, eta_2700, eta_3000]), ...
    max([eta_1800, eta_2100, eta_2400, eta_2700, eta_3000]), 10);

[C, h] = contour(Md, Pr, Eta, contour_levels, 'b', 'LineWidth', 0.8, 'LineStyle', '--');
clabel(C, h, 'FontSize', 8, 'Color', 'b'); % Add contour labels

% Labels and title
xlabel('Mass Flow Rate (kg/s)');
ylabel('Pressure Ratio');
title('Compressor Map');

% Legend (only for data points + surge line)
legend([scatter_handles, scatter(mdot_surge, PR_surge, 80, 'k', 'filled')], ...

```

```
{'1800 RPM', '2100 RPM', '2400 RPM', '2700 RPM', '3000 RPM', 'Surge Points'}, ...  
  'Location', 'Best');
```

```
hold off;
```